

Cross-Room Few-Shot Adaptation

CS60055 Ubiquitous Computing | Hackathon Challenge 2

Environment-Aware, Label-Efficient Activity Recognition from WiFi Channel State Information

1. The Scenario

A WiFi activity model can change when the physical environment changes. Walls, furniture, geometry, radio conditions, gain state, and session conditions shape the channel response. A model trained in one environment may therefore carry environment-specific information into activity recognition.

A home-care provider wants to deploy WiFi sensing in a new indoor environment after only a short setup visit. During that visit it can collect a small number of labeled examples of activities, but it cannot afford to label every activity in every environment.

This hackathon turns that deployment question into a controlled benchmark using the CSI Human Activity dataset. The scored core consists of two recorded environments with the same hardware platform and nominal CSI dimensionality but different rooms and sessions. The benchmark asks whether a team can reconstruct a clean CSI representation from the source files, learn useful representations without activity labels, and transfer activity knowledge across rooms with very few target-domain labels.

2. What You Will Build

Your submission is a complete Python sensing-and-learning pipeline. The model is only one part of the task. Every team must implement its own data ingestion, CSI reconstruction, conversion to amplitude, quality control, common-format export, normalization strategy, windowing, representation learning, adaptation, and inference.

- A Python data-ingestion pipeline. Read the challenge-provided source files, inspect their structure, and convert the two room datasets into a common numerical representation without using MATLAB or the source authors' code.
- A recorded-environment fingerprint. From a fixed CSI amplitude window, predict whether the observation came from Room A or Room B. This is a routing component for the challenge, not a claim of universal room recognition.
- A self-supervised CSI encoder. Learn a representation from allowed unlabeled recordings without using activity or room labels. The representation should remain useful when moving between the two core environments.

- Cross-room few-label adaptation. When one room is the target, use only K labeled target captures for target-exclusive activities and recognize the shared SIT, STAND, and WALK activities without target labels.
- The central question is label efficiency: how does performance change as K increases from 1 to 5 to 10 to 20, and how much do self-supervised pretraining and environment-aware routing contribute?

3. The Data

The challenge uses the IEEE DataPort CSI Human Activity dataset from Frankfurt University of Applied Sciences, associated with Schäfer et al., Applied Sciences 2021 (<https://ieee-dataport.org/open-access/csi-human-activity>). The original work used Nexmon-based CSI extraction in three indoor line-of-sight experiments.

Rooms A and B use the same hardware platform, bandwidth, and nominal CSI dimensionality. Because the experiments were recorded in separate sessions, the benchmark is described as cross-environment transfer rather than a pure physical-room-isolation study. The benchmark does not claim subject-disjoint generalization or generalization to an arbitrary unseen room.

4. How the Dataset Will Be Stored and Distributed

We'll share a Google drive link with the students containing the data and documentations. The participant package must not include the source authors' MATLAB directory or the Experiment-3 LSTM notebook.

The public participant package should preserve the source dataset structure closely enough that teams must genuinely solve the data-engineering problem. The following structure is part of the challenge specification:

```
CSI-HAR-Hackathon/
├── Experiment-2/
│   ├── ReadMe.pdf
│   └── realdata/
│       ├── input_data/
│       │   └── 30ms/
│       │       └── csidata/
│       │           ├── fall/           # raw .pcap captures
│       │           ├── sit/
│       │           ├── sitdown/
│       │           ├── stand/
│       │           └── standup/
```

```

|
|           └─ walk/
|           └─ validation/ # supplied validation captures; 25 per
class
|
|           └─ fall/
|           └─ sit/
|           └─ sitdown/
|           └─ stand/
|           └─ standup/
|           └─ walk/
|
└─ Experiment-3/
    └─ ReadMe.pdf
    └─ Data/
        └─ Empty_Amp_*.csv
        └─ Lying_Amp_*.csv
        └─ Sitting_Amp_*.csv
        └─ Standing_Amp_*.csv
        └─ Walking_*.csv

```

Observed archive counts used when designing the benchmark are shown below. The counts are included so that teams can verify that they downloaded the intended data package; they are not permitted to use the source authors' training/test split.

Room	Format	Observed total	Development / reserved information
A / Experiment 2	Raw .pcap	1,084 captures	934 development captures + 150 validation captures; validation contains 25 per class
B / Experiment 3	Amplitude .csv	1,201 captures	No validation folder in the supplied archive; the teaching team creates a capture-disjoint held-out set before participant release

Important: remove archive housekeeping files such as `__MACOSX/` and `.DS_Store` from the participant copy. Do not commit the dataset to GitHub. Students should work from the shared data location and maintain only their code, configuration, derived local files, and results in the Github repository.

5. The Student-Owned Python Preprocessing Pipeline

The challenge does not prescribe a single preprocessing algorithm. Instead, it specifies the required interfaces and provides staged hints. Your implementation must be in Python from raw input to final model input. You may use general-purpose Python packages for packet parsing, numerical computing, signal processing, and machine learning, but you must not translate, port, or reproduce the source authors' MATLAB functions.

5.1 Hint 1 - Start by inspecting the source files

What to inspect

Determine file type, packet/row count, payload structure, sequence ordering, activity label source, subcarrier dimensionality, and whether timestamps are available. Do not assume that a folder name such as "30ms" is a capture duration.

5.2 Hint 2 - Decode Experiment 2 PCAPs in Python

Your task

Experiment 2 contains raw Nexmon CSI from the BCM4366c0 platform at 80 MHz. A valid observation contains a chip-specific packed CSI representation. Your Python code must locate valid CSI payloads, reconstruct the complex CSI values, and then derive a real-valued amplitude representation.

Useful checks

The core Experiment-2 representation should resolve to 256 CSI subcarrier values per valid observation. Inspect invalid/empty CSI observations and preserve temporal order.

Library hint

A general-purpose PCAP parser is allowed. The challenge does not require you to implement Ethernet/IP/UDP decoding from first principles. What matters is that the CSI reconstruction and subsequent signal processing are your own Python code.

5.3 Hint 3 - Bring Room B into the same intermediate form

Room B starts later in the pipeline

Experiment 3 CSV files already contain amplitude values with 256 columns. Read them in Python and make their schema match the amplitude representation produced from Experiment 2.

Common intermediate form

A recommended common intermediate file is one CSV per capture with 256 amplitude columns: one row per valid CSI observation and columns named consistently (for example, subcarrier_001 ... subcarrier_256). The exact column naming is up to you; the numerical content and ordering must be correct.

Why this matters

Once both rooms are represented as the same 256-column amplitude sequence, the downstream pipeline no longer needs to know whether the original source was PCAP or CSV.

5.4 Hint 4 - CSI quality control

Check validity before modeling

Look for malformed packets/rows, missing values, zero-information CSI, duplicate observations, and non-finite amplitudes. Document what you reject and why.

Do not assume denoising is mandatory

The source code contains a wavelet-denoising function, but the main Experiment-2 reader does not call it in the active path. You may investigate denoising or filtering as your own design choice, but do not copy the source implementation.

5.5 Hint 5 - Subcarriers

Use the 256-subcarrier representation for the core benchmark

The supplied Experiment-2 processing path ultimately returns a 256-column amplitude matrix. Do not silently reduce the representation because the source code happened to construct a null/pilot removal list; verify what your own pipeline actually returns.

5.6 Hint 6 - Build a capture-level common dataset

Atomic unit

Treat the original capture as the fundamental unit of splitting. Your intermediate representation should preserve capture identity internally so that no capture is split between development and validation.

Suggested student-side workspace

Keep raw data read-only and write derived data into a separate directory. A sensible layout is shown below; you may use another layout as long as it is reproducible.

```
student_workspace/  
├─ raw/  
│   ├─ experiment2/  
│   └─ experiment3/  
└─ intermediate_csv/
```

```
|   |─ room_A/
|   |─ room_B/
|─ windows/
|   |─ development/
|   |─ local_validation/
|─ src/
|   |─ pcap_reader.py
|   |─ csi_decode.py
|   |─ amplitude.py
|   |─ cleaning.py
|   |─ windowing.py
|   |─ normalization.py
|   |─ ...
|─ configs/
|   |─ experiment.yaml
```

5.7 Hint 7 - Windowing

Official model input

The final benchmark input is a fixed window of 150 consecutive valid CSI observations by 256 subcarriers: (150, 256). The 150-observation size is a challenge design choice inspired by the supplied Experiment-2 processing workflow; it is not a statement that every raw capture is exactly 150 observations long.

Your design still matters

Implement and document a deterministic way of selecting the 150 observations from a capture. Do not use labels or statistics from validation captures when designing or fitting the preprocessing pipeline. Do not mix observations from different captures into one window.

5.8 Hint 8 - Normalization

Choose and justify

Possible strategies include per-capture normalization, training-set statistics, per-subcarrier scaling, robust scaling, or another defensible transform. There is no single mandated normalization method.

Leakage rule

Any learned normalization parameters used for training must be fitted from development data only. During validation, apply the already-chosen preprocessing to validation captures without refitting the model on their labels.

5.9 Hint 9 - Preserve a reproducible provenance trail

Provenance trail

For every derived window, be able to explain which original capture produced it and which preprocessing configuration produced it. This provenance should remain auditable in your repository even after you reorganize files for modeling.

6. Pipeline Independence: What Is Allowed and What Is Not

A team may independently arrive at the same scientific operation described in the paper. What is not allowed is reproducing the authors' implementation as the team's preprocessing solution. Every team member should be able to explain the purpose and behavior of every preprocessing stage.

Allowed	Not allowed
IEEE DataPort dataset and the challenge paper	Source authors' MATLAB implementation
Official/public documentation for Nexmon and CSI formats	Direct translation/port of fileReader.m, createLSTMDataset.m, CSI_signal_processing.m, denoise.m, or other source scripts
General-purpose Python libraries for PCAP parsing, NumPy/SciPy, pandas, scikit-learn, PyTorch/TensorFlow, plotting	Source authors' processed .mat files or derived training artifacts
Public research papers and standard signal-processing/ML techniques, with citations	Using another team's preprocessing or model code
AI tools, subject to the course AI-use disclosure rules	Submitting generated code without understanding, attribution, or disclosure where required

7. The Three Tasks

Task 1: Recorded-Environment Identification

Given a 150×256 CSI amplitude window produced by your Python pipeline, predict whether it came from Room A or Room B. The classifier must operate on the common numerical representation, not on file extension, directory structure, filename pattern, capture length, packet count, or any other source metadata.

Report room accuracy on the fixed validation windows. You may additionally study shorter windows, but all headline numbers in the report must use the official 150-observation validation windows.

Task 2: Learn Without Labels

Train a self-supervised encoder on development recordings from both core rooms without using activity labels or room labels. The validation set is used only for the downstream evaluation and reported results.

Freeze the encoder and fit a linear classifier using labeled development data. Score the frozen representation on the fixed validation set. Compare against a handcrafted-feature baseline and the same network architecture with random weights.

Task 3: Cross-Room Few-Label Adaptation

Target room	K-labeled target captures	Recognize without target labels
Room A	SIT-DOWN, STAND-UP, FALL	SIT, STAND, WALK (learned from Room B)
Room B	EMPTY, LYING	SIT, STAND, WALK (learned from Room A)

For each transfer direction, one core room is the target and the other is the source. The source room provides all labels from its development captures. The target room provides only K labeled capture-level support examples for the activities observed only in that target room. Shared SIT, STAND, and WALK activity labels in the target room are withheld from support.

For each $K \in \{1, 5, 10, 20\}$, create a support set from the target room's DEVELOPMENT captures. K means K distinct original captures per target-exclusive activity. A single original capture is the atomic unit for splitting; multiple windows from one capture do not count as multiple shots.

Evaluate every K setting on the fixed VALIDATION set for the target room. The validation set is never used as support, training data, normalization-fitting data, or for any other form of model fitting. Report the results for all K values and for both transfer directions.

8. Evaluation Protocol and Leakage Controls

Capture-level isolation:

The original capture is the atomic unit of the development/validation split. If a capture produces multiple windows, all of those windows stay in the same split.

Validation is for evaluation:

Use the fixed validation set for all reported experimental results. It may be used for model comparison and design decisions, but validation results must be described as validation results, not as unseen-test accuracy.

Target-label isolation:

For Task 3, target-room support examples must come only from the target room's development captures. Validation captures are never eligible for support.

No metadata shortcuts:

Do not use filenames, folder names, file extensions, capture length, packet count, directory structure, or other source metadata as features for room or activity prediction.

No query matching:

Do not identify a validation sample by matching it to a known capture or by exploiting how validation windows were generated.

Leakage-safe preprocessing:

Any preprocessing parameter learned from data must follow the split rules. A validation-derived statistic may be used only for validation analysis; it may not be used to train or fit the final model.

9. Development and Validation Structure

The challenge uses a fixed Development/Validation structure, but the two source experiments provide that structure differently.

Experiment 2 already contains a validation/ directory with 25 captures per activity. Use those captures as the Room-A validation set. Do not move them into development.

Experiment 3 does not contain a validation/ directory. Teams must create one using 25 captures per activity, selected at the original-capture level with the public random seed 42. All remaining captures form the Experiment-3 development set.

Experiment-3/

```
|— development/
|   |— Empty_Amp_*.csv
|   |— Lying_Amp_*.csv
|   |— Sitting_Amp_*.csv
|   |— Standing_Amp_*.csv
|   |— Walking_*.csv
|
|— validation/
```

```

├─ Empty/          # 25 captures
├─ Lying/          # 25 captures
├─ Sitting/        # 25 captures
├─ Standing/       # 25 captures
└─ Walking/        # 25 captures

```

For Experiment 3, sort the candidate capture filenames deterministically before sampling with the stated seed so that every team constructs the same validation set. Validation is created once at the capture level; do not resample it during the project.

For both rooms, the remaining captures are development data. Teams may further divide development data internally for training, source-model fitting, local checks, or support selection, but the official reported results for all tasks are computed on the fixed validation sets.

For Task 3, select K distinct target-room development captures per target-exclusive activity. Run K = 1, 5, 10, and 20. Evaluate on all eligible validation captures of the target room and report the results for both $A \rightarrow B$ and $B \rightarrow A$.

The teaching team may run the submitted pipeline on a small unseen set during the final demonstration or reproducibility check. This is an additional check only; the technical report is based on the validation results described above.

Student workflow

Source data (read-only)

```

|
├─ Experiment 2: .pcap + supplied validation/
|
└─ Experiment 3: amplitude .csv

```



STUDENT PYTHON PIPELINE

```

|
PCAP/CSV ingestion and CSI processing
|
common 256-column form
|
quality control
|
normalization
|
150 × 256 windows

```

|
SSL / room / HAR model
|
VALIDATION RESULTS

10. Input and Output Contract

Model input

window_XXXX.npyshape: (150, 256)dtype: float32content: CSI amplitude produced by the team's Python pipeline

The 150×256 array is the model-facing representation. Teams are responsible for producing it from the source captures. The common intermediate CSV is a student-generated artifact used to make the two source experiments compatible; it is not supplied as a solution.

Student output for experiments

Each reported experiment should record at least:experiment, target_room, K, metric, valueFor final predictions, use:window_id, room, label, confidence

For Task 1 and Task 2, report the required validation metrics. For Task 3, report $A \rightarrow B$ and $B \rightarrow A$ results for $K = 1, 5, 10$, and 20 , including the primary macro-F1 and the target-exclusive macro-F1.

The same Python preprocessing pipeline must be used for development and validation data. You may cache intermediate representations locally, but the repository must contain the code required to regenerate them from the source data.

11. Metrics and Required Figures

The headline activity metric is macro-F1. Also, report balanced accuracy. Room identification is scored by accuracy. For Task 3, report the validation result for each K and for both transfer directions.

- Label-efficiency curve: macro-F1 against $K = 1, 5, 10, 20$ for each target room, with separate curves for target-labeled and transferred shared activities.
- Encoder diagnosis: 2-D UMAP/t-SNE embeddings colored once by room and once by activity. Treat it as qualitative, not as the sole representation metric.
- Environment fingerprint: room accuracy against window length, using the fixed validation capture pool.
- Routing contribution: compare predicted-room routing, oracle-room activity, and the same adaptation model without the routing stage, all evaluated on validation data.
- Confusion matrices: one per target room at $K = 5$, with precision, recall and F1 per activity.

12. Known Limitations and Required Interpretation

1. Room versus session confound: Room A and Room B were recorded in separate sessions. A classifier may therefore exploit session-specific effects as well as physical environment.
2. Activity versus location confound: the original Experiment-2 collection used different spatial zones for some activities. Location and activity can therefore be correlated.
3. Two core environments only: the benchmark evaluates $A \leftrightarrow B$ transfer. It does not test a completely unseen room.
4. No subject-disjoint claim: the supplied materials do not establish a participant-disjoint split across the two core rooms. The benchmark is capture-disjoint, not subject-disjoint.
5. Original-paper results are not directly comparable: the paper's experiments and random 80/20 workflow answer a different question than this capture-disjoint cross-room benchmark.
6. The source archives are asymmetric: Experiment 2 begins as raw PCAP while Experiment 3 is already amplitude CSV. The student pipeline must bring both into a common numerical representation before downstream learning.

13. Relationship to the Original Paper and Source Implementation

The benchmark uses the same underlying dataset but deliberately defines a new student-owned Python preprocessing pipeline and a new cross-room evaluation protocol.

Experiment 2 uses BCM4366c0 at 80 MHz with a 256-point CSI representation and chip-specific packed-CSI decoding before complex CSI and magnitude extraction.

The supplied source code contains implementation details that may help the teaching team verify the dataset lineage, but those implementations are not part of the participant starter package.

The original paper's random train/test evaluation is not the protocol used in this hackathon. Students must report results using the challenge's Development/Validation structure.

These facts are provided for context and verification, not as a recipe to reproduce the authors' implementation.

14. Rules, Repository, and Deliverables

Groups and repository

Work in assigned groups of 2-3 members. Every member is expected to contribute. Maintain one GitHub repository with incremental commits throughout the challenge.

The repository must contain Python source code, a README, a reproducible environment specification, the pipeline configuration, the `run_episode.py` entry point, and the technical report. Do not commit the dataset itself.

Academic integrity and AI use

A runnable Python implementation of the complete preprocessing and learning pipeline. The teaching team may use it on a small unseen set during the final demonstration/check.

Required deliverables

GitHub repository with Python preprocessing, model code, setup instructions, and reproducibility scripts.

Technical report of about 8-10 pages, including the preprocessing pipeline diagram, design rationale, implementation, results, required figures, contribution statement, and AI-use disclosure.

Runnable end-to-end system that can process a challenge episode and write predictions in the required format.

A short demonstration/presentation, including a live run on an episode supplied by the teaching team.

15. Where to Start Reading

- CSI/Nexmon data formats and Broadcom packed-CSI decoding documentation.
- CSI-based indoor localization and environment fingerprinting: static multipath and frequency-selective fading.
- Self-supervised learning for CSI and time series: contrastive learning, masked reconstruction, and temporal consistency.
- Few-shot learning: prototypical networks, nearest-neighbor classification, and small-head adaptation on frozen representations.
- Unsupervised domain adaptation: per-domain normalization, feature alignment, and test-time adaptation.
- Open-set recognition: distance- or uncertainty-based rejection for the optional UNKNOWN setting.

16. Source References

1. IEEE DataPort, CSI Human Activity dataset:
<https://ieee-dataport.org/open-access/csi-human-activity>.

2. Schäfer, J.; Barrsiwal, B.R.; Kokhkharkova, M.; Adil, H.; Liebehenschel, J. "Human Activity Recognition Using CSI Information with Nexmon." *Applied Sciences*, 2021, 11(19), 8860. <https://doi.org/10.3390/app11198860>

Design intent: students should spend meaningful effort understanding the CSI data, constructing the preprocessing pipeline in Python, preventing leakage, and then designing the representation-learning and few-shot adaptation method. The challenge gives constraints, interfaces, and hints, not the source authors' solution.